Serial Decode — principles of operation

Ian Ameline · version 1.42 · MIT licence

How the app turns samples into bytes, and how it recovers the bit rate and the frame format from a line that declares neither. MANUAL.md covers using it. REFERENCE.md holds the bench measurements — the tolerance envelope, the verified rates, the endurance figures.

The problem

An asynchronous serial line carries no clock and no metadata. Bit rate, data width, parity and signalling polarity are agreements between two devices; the wire shows none of them.

Two properties make the job tractable. Every pulse lasts a whole number of bit times, which turns the bit rate into a fit. And a wrong frame format puts the stop-bit sampling point on a data bit, which reads as space about half the time and raises a framing error — so the format turns into a search that the data itself arbitrates.

The pipeline

1 samples -> 2 levels -> 3 threshold -> 4 edges -> 5 pulse widths -> 6 bit time -> 7 frames -> 8 bytes

Each stage reduces the data sharply. A 20 000-sample capture becomes a few hundred edges. The framer then reads ten to twenty computed positions per byte and never scans the samples again, so decode cost follows the byte count rather than the capture length. That enables a brute-force format search.

Stages one to five are protocol-independent and live in `tsp/serial_core.tsp`. Framing and the format search live in `tsp/uart_decode.tsp`. Recordings longer than memory are windowed by `tsp/chunk_decode.tsp`.

Acquisition

The app digitizes voltage on the 10 V range, which has the widest digitize bandwidth the instrument offers — 440 kHz, against 17 kHz on the 1000 V range — and takes 20 000 samples per capture.

Sample rate trades resolution against duration, and only one of the two is known before the capture. At a fixed 1 MS/s, 20 000 samples span 20 ms whatever the line speed, so a slow line yields *less* text: 19 bytes at 9600 baud. The app therefore captures twice. The first pass runs at 1 MS/s and measures the bit rate. The second re-captures at the lowest listed rate still giving 8 samples per bit, which turns those 19 bytes into about 240. A forced rate skips the first pass, because the right sample rate is then known in advance.

The probe pass free-runs and retries down a ladder — 1 MS/s, 100 kS/s, 10 kS/s — each step covering ten times as long for the same sample count. A 20 ms window is shorter than one frame at 300 baud, so identical looks at a slow line can all land in its idle stretch. The probe needs only a bit time, so where it starts does not matter; the real capture keeps the operator's trigger setting.

The ladder is built from rates that divide 66 MHz exactly, because the sample clock is 66 MHz with an integer divider rounded up. Three exceptions — 160, 320 and 640 kS/s — because they are what 19200, 38400 and 76800 need, and without them those three commonest bit-banged rates snap up a whole step

and return 192, 192 and 153 bytes per capture instead of 240 each. They cost 8.32, 8.30 and 8.26 delivered samples per bit, all above the 8 the selector asked for. The true rate is re-derived from the buffer's own timestamps after every capture, because the readback lies for any rate the hardware cannot synthesise.

Two logic levels, found by density

The two logic levels are the two densest amplitudes in the capture. Working outward from density beats working inward from the extremes.

Density needs no budget. A logic level holds hundreds of samples in one or two bins, while spikes of differing amplitude scatter one or two samples per bin. The minority logic level wins on count however many spikes there are.

`sig_pair` bins the amplitudes at 16 samples per bin, 512 bins at most, and finds the tallest bin. It then measures that hump's extent outward to a fifth of its own height rather than assuming one, because the separation between two levels cannot be a fixed voltage. Noise widens the humps: requiring a fixed gap instead put both "modes" 14 bins apart on opposite flanks of the *same* hump as soon as the spread exceeded 0.1 V. The second level is the tallest bin lying outside that hump and at least 0.1 V from its peak, with its own hump measured the same way.

Each level is the mean of the samples inside its own hump, accumulated per bin during the histogram pass. Membership is the bin index, not a re-derived voltage band. That round trip loses: on a clean 0 to 3.3 V capture it put a hump's lower edge 4.44e-16 V above the very sample that defined the hump, so no sample fell inside the band and a textbook waveform read as a dead line — at 15 bytes but not at 13, 14, 16 or 17, which at the bench looks exactly like a probe on the wrong pin.

The threshold is the midpoint of the two levels and the hysteresis is 15 % of the swing. `modal`, the share of samples sitting at either level, grades the wire: a clean line reads about 0.95, below 0.50 the panel warns of drift, and below 0.15 the app refuses — two humps holding almost none of the samples are noise, not levels. A swing under 0.1 V is refused outright rather than split down the middle to manufacture edges out of ADC noise.

`sig_family` names the logic family from the levels — RS-232, 5 V TTL, 3V3 CMOS, 1V8 CMOS — which is the quickest way to spot RS-232 mistaken for TTL.

Edges

`sig_edges` walks the capture with the hysteresis band and records one fractional sample position per transition. Because the trip is detected only after the signal has passed `thr ± hyst`, the threshold itself was crossed one or more samples earlier — several on a slow RS-232 edge — so the code walks back up to 64 samples to the pair straddling `thr` and interpolates between them.

Interpolation costs one divide per edge and keeps edge positions independent of the hysteresis setting. It has to: the bit-time fit reads these positions as timing, and at 115200 baud and 1 MS/s there are 8.68 samples per bit, so a grid-quantised edge carries up to ±5.8 % of a bit time — enough to walk the mid-bit sampling point into the next cell across a ten-bit frame.

Bit time

The pulse widths are the gaps between consecutive edges. Every one is an integer multiple of the bit time, so the fit passes through the origin: given widths `w[k]` and integer multiples `m[k]`,

```
T = sum(w*m) / sum(m*m)
```

over the widths lying within 35 % of a multiple, with `m` at most 12. A longer gap is inter-byte idle, carries no timing information, and must not drag the fit. The loop re-derives the multiples from each new `T` and stops after six passes or when `T` moves by less than a part in a billion.

Using every pulse beats using the shortest. One short pulse carries the full sample quantisation error; a ten-bit gap divided by ten carries a tenth of it, and the fit weights long gaps accordingly.

Seeds come from the smallest widths, but only corroborated ones — another width within 25 %, thinned to 30 % apart, four at most. Corroboration is not optional. A sub-bit glitch produces one isolated short width, and a fit seeded from it can score *higher* than the truth, because a shorter trial period reclassifies the contradicting long widths as idle and drops them from its own coverage denominator. Scoring cannot reject that; requiring a neighbour does, because real bit-time widths arrive in dozens and glitches arrive alone.

Quality combines tightness with coverage:

```
q = (1 − mean_error/T) * (widths that fit / widths eligible to fit)
```

Coverage is what rejects a seed at twice the true bit time. The real one-bit pulses stay inside `maxmult * T` so they count as eligible, but they land half a period from any multiple and fail the tolerance.

The result is a proposal as opposed to an answer. Pulse statistics cannot always find the bit time, because the widths can share a common factor: all 0x00 at 8N1 with a two-bit gap holds only nine-bit and three-bit runs, and best-fits three bit times. Framing settles that, further down.

Baud rate

The rate is `fs / T`, with `fs` measured. `sig_snap` moves it to the nearest of 25 standard rates when within 2 %, because real devices run at standard rates. Inside 1.25 % the snap is reported as firm; outside it the panel marks the rate approximate, since the measurement and the label then disagree by more than the fit's own accuracy. A rate close to no standard rate is reported as measured.

`baud_round` tidies a non-standard rate before the app locks it, taking the coarsest step that moves it less than 1 % — nearest 100 baud above 10 kBd, else nearest 10, else nearest whole baud. A locked rate has to read like something an operator could have typed: 16099 locks as 16100. A rate already on the standard ladder is returned untouched, which protects MIDI's 31250 from a 100 baud step.

Polarity

The mark level is the one holding the longest single run. That is correct whenever the capture holds an inter-byte gap, and it separates idle-high TTL from idle-low RS-232 without being told.

A run is evidence of idle only if data could not have produced it. At 8N1 the longest run data can make is nine bit times — a start bit plus eight zero data bits, or eight ones plus the stop bit — so ten is the smallest run that must be idle. The app demands 10.5, because both terms of the comparison carry error: the run is measured on a jittered edge, and one bit time is estimated as the 10th percentile of the pulse widths rather than their minimum, so that a single narrow glitch cannot halve it.

The margin is supported by evidence - Measured at the bench on real 9600-baud ASCII, `run0` was 63 samples and `run1` 42, so longest-run said the line idled low when it idles high. Both runs were data: 63 samples at 10.42 samples per bit is the start bit of a space, 0x20, running together with five zero data bits. The window held no idle at all.

Where the runs carry no information, the decision falls to the levels — a fact about signalling rather than a guess about this capture. A single-supply logic line marks high, and one straddling ground is RS-232 at line levels and marks at its negative level. The app flags that prior as weak, and a rival polarity must then beat it by four times the ordinary margin. It has to, because the inverted reading of ASCII frames just as cleanly as the correct one — ASCII's top data bit is always 0, which puts a rising edge exactly nine bit times after every start bit, a perfectly periodic fake start edge. Three consecutive captures of one 9600-baud line: the first decoded correctly, the second returned 18 frames at the wrong polarity with zero errors, and the third inherited it. Self-consistent, confident, and every byte wrong. That failure is the one this decoder is built around.

One frame

Bit cell `k` of a frame whose start edge sits at `t0` spans `[t0 + k*T, t0 + (k+1)*T)`, so its centre is at `t0 + (k + 0.5)*T`. Cell 0 is the start bit; data bits follow LSB first, then the parity cell if the format has one, then the stop bits.

At 6 samples per bit or more the sampler majority-votes three taps spanning 40 % of the cell instead of reading one. A single sample is one ADC conversion, so a noise excursion at that instant flips the bit; three taps need two wrong together. The centre tap defines the bit and breaks ties, and its absence defines a truncated frame — tested first and unconditionally, because a frame running off the end of the capture must report truncation rather than decode from whatever samples remain and fabricate a trailing byte.

A failed start bit, a parity mismatch and a failed stop bit are distinct outcomes, and parity and framing failures still return the value. A byte with a bad stop bit is usually the right byte at the wrong baud rate.

Sequential framing

Framing is sequential, not per-edge. Inside a byte a 1-to-0 data transition is indistinguishable from a start bit; only knowing where the current frame ends separates them. So the framer consumes a frame, jumps past it, and then looks for the next start edge. Treating every falling edge as a start bit produces plausible garbage.

The first frame has no predecessor, so it anchors on an edge preceded by at least 1.5 bit times at mark — a mark run that long cannot occur inside a frame. Failing that, on a gapless stream or a capture that begins mid-byte, the framer takes the first candidate edge and accepts that the first frame may be misaligned.

Three details keep that walk functioning:

- **A failed start bit is a false trigger, not a corrupt byte.** The edge was not a frame boundary — usually an impulse on an idle line, back at mark by mid-bit. Counting it as an errored byte would leak the spike count into the error count, which is what the format search scores on, so impulse noise could tip the search onto the wrong format. Skipping it also leaves the resynchronisation point alone, so a false trigger cannot desynchronise the frames behind it.
- **Errors at the capture boundary are not signal errors.** A capture of a busy line is sliced through a frame at each end, and a gapless stream resynchronises a few frames in. Without the exclusion, every healthy 9600-baud capture reported one or two errors in the first row and coloured it red. An error count

that is never zero on good data is decoration, not a fault indicator. So the first three frames and the last one are excluded from the count.

- **A mid-byte start corrupts the head and nothing after it.** The framer records the first genuine idle — a pitch of two frame times or more — and marks the region before it suspect, but only on evidence: errors confined to the head. A capture that merely began at a message boundary has that idle too and no errors, so nothing is marked. Bytes in a flagged head count in full, because wrong bit boundaries can pass parity and stop by luck and hand back a plausible wrong byte.

The format search

Data width, parity and stop bits are not derivable in closed form, so the app decodes under every plausible format and keeps the fewest-error result. By default that is 2 widths × 3 parities × 2 polarities — twelve passes, each reading ten to twenty positions per byte.

Search order doubles as tie-break order, so the commonest bench format wins when two formats explain a capture equally well. A decode ranks as `ngood − 3∗nbad`: errors weigh heavily, because the whole premise is that a wrong format shows up as framing errors. Forty bytes with twelve errors is not a better description of a line than thirty with none.

A plain `>` comparison would be wrong, because several formats are genuinely indistinguishable on the wire and the shorter frame of such a pair always finds more frames in the same capture. Repeated 0x55 at 8N1 with a two-bit gap has a 12-bit period, and three 8-cell 6N1 frames tile two real frames exactly, so 6N1 decodes 9 error-free frames where 8N1 decodes 6. Both are complete, consistent descriptions, and no signal processing separates them. So a rival must win by more than 60 %, with an absolute floor of 2 points to stop noise deciding on a handful of frames. Being proportional, the rule behaves the same on a 6-byte capture and a 600-byte one, and real evidence clears it easily: genuine 5N1 scores 8 against 8N1's 1.

rule	why
only 7 and 8 data bits by default	a wider frame launders damage into a data bit, so its error count *falls* as the answer gets worse, and every byte comes back 256 too large
5 and 6 biased against by 8×	they tile longer frames exactly, which produces a dead tie decided by search order, while genuine rare-width traffic wins by a ratio
9 data bits never searched	the bias goes inert exactly when a laundering width can win, because a damaged incumbent's own score is already zero. Honoured when forced
one stop bit only	a second stop bit is indistinguishable from a bit time of idle, which the framer already tolerates, so `nstop = 2` can never rescue a capture that `nstop = 1` fails. Honoured when forced

Formats whose score came within the margin of the winner's are reported as ambiguity on the note line, once the capture holds at least 8 good frames. Below that almost every format lands inside the floor, and "also fits: 9N1 6N1" is true of the capture and useless as a report.

A forced field narrows the candidate list rather than sitting in a variable, so Parity, Stop Bits and Polarity each take effect alone. A forced polarity ends the contest instead of entering it.

Framing arbitrates the bit time

The width fit has two blind spots that pulse statistics cannot cover, so the app scores candidate bit times by how well they *frame* and keeps the winner. Candidates are the fit rescaled by a ladder of small integer ratios, plus the median pulse width divided by 1 to 6.

The ladder is asymmetric — integers to 10, reciprocals only to 1/6 — because the two error directions are not equally likely. The fit finds the greatest common bit period of the observed pulses, so it errs *short*; erring long by a factor k needs every pulse in the capture to be an exact multiple of k bit times, which stops being constructible inside a ten-bit frame past k = 6. The median candidates cover the one failure the ladder cannot: forty 2-sample spikes corroborate each other, the fit lands at 0.95 samples against a true 10.42 — a ratio of 11, which no ladder will ever carry — and the median cannot be moved that way because spikes are a minority of the widths however many there are.

Each candidate is probed with 3 widths × 3 parities × 2 polarities, over at most 4000 samples from the first edge. The final decode still runs over everything; only the ranking is truncated.

Rescaling is gated rather than unconditional, because halving a bit time roughly doubles the frames found, so every frame-counting score is biased toward a shorter period. On a repeated 0x55 the true bit time frames 6 bytes and a quarter of it frames 30, all error-free, and every rescaling of an alternating pattern is self-consistent. Six gates hold that off, in the order the code applies them:

- **Rescale at all only when the fit is plausible or demonstrably corrupt**, and one number separates those. In async serial the median pulse is two bit times, because a ten-bit frame plus a couple of idle bits produces runs whose middle value is about two. Measured median over fit is 2.0 for every healthy capture tried — 9600 clean, 250 kBd, 285 kBd, 921 kBd and every drift case — and 3.0, 11 and 13 for the three spike-corrupted fits. A fit out of range *and* consistent with its own widths is telling the truth, and the truth is out of range; rescaling it would manufacture a plausible-looking wrong answer. A fit grossly inconsistent with its widths is itself the error, and rescaling is the right move.
- **A candidate must explain the pulse widths too**, to within 80 % of the best candidate's fit quality. That is the second, independent piece of evidence, and it breaks the one ambiguity framing alone cannot: on a badly corrupted 9600-baud capture, 7N1 at 7200 baud frames 54 of 54 bytes with zero errors while the truth frames 40 with 15. No frame-error score can prefer the truth, and pulse-width consistency is decisive — 0.855 against 0.497 — because the real widths are multiples of the real bit time and of nothing else.
- **A candidate must not be a sub-multiple.** Every pulse that is a multiple of `T` is also a multiple of `T/2`, so quality cannot separate them while the halved reading finds twice the frames. The odd multiples can: varied data holds single- and three-bit runs at the true bit time, and at half of it every multiple is even. Measured over 368 640 paired decodes, the odd fraction is 0.011–0.013 at a halved bit time against 0.587–0.604 at the true one.
- **A candidate must not claim the shortest pulse on the wire is several bits long.** This covers the odd divisors the odd-multiple test structurally cannot, since an odd divisor leaves every multiple's parity as it was. A ten-bit frame opens with a one-bit start bit, so the 5th-percentile width is 1.00 bit times at the truth and 5.00 at the 1/5 candidate that displaced it.
- **A fit that snapped to a standard rate and scored above zero is kept.** A rescaling displaces it only when the fit is itself failing — at least a tenth of its frames bad — and then only by a factor of three. A clean fit is never displaced.
- **Failing that, a rescaling that frames nothing may still not displace an admissible snapped fit.** A score at or below zero means the errors outnumber the good frames threefold, so ranking two such candidates against each other is ranking noise. Measured on a LIN bus with no break delimiter, where

the merged break and start bit destroy the framing: the true 19200 scores −20 and its own half scores −15, so the half wins on −15 > −20 and is reported as a confidently snapped 38400.

The two sub-multiple gates apply only to candidates, never to the fit, and only when the fit snapped. When it did not snap it is not a good fit, and a rescaling is the only route to the truth: repeated 0xE0 at 8N1 is all six-bit runs, so the fit lands on 1600 baud and the truth is the 1/6 candidate, whose multiples are all even and which an ungated test would reject.

Where two readings are both legitimate, the app says so rather than choosing silently. Eight 0x00 frames at 9600 7N1 with a one-bit gap read as four 0x08 bytes at 4800 8N1 with zero errors, and the two encodings are bit-for-bit the same waveform. Which device is on the wire cannot be recovered from it, so the alternative goes on the note line.

Two refinements the search cannot make

Collapsing an always-one top bit. A data bit that never changes carries no information, and in the top position it is indistinguishable from the stop bit that must follow it: N data bits with the MSB always 1 describe exactly the same cells as N−1 data bits with the stop bit one place earlier. Error scoring cannot separate them, so the app walks the width *down* while every error-free frame has its top bit set — 8 to 7 by default, as far as 5 under "Auto (any width)". The masking matters: once a bit is dropped the next test must look at the remaining value, because a 6-bit stream read as 7 bits has values d + 64, which are always at least 32 and so always look like another constant 1.

Only an always-*one* bit qualifies. An always-zero MSB is ordinary 7-bit ASCII in an 8-bit frame and is left alone, and seven data bits plus *mark* parity genuinely do put a constant 1 there — which is why the collapse is reported rather than silent. A stuck top bit is also not evidence of a wide device: nine-bit formats exist for multidrop address marking, where the ninth bit varies by design.

The refinement runs only at parity None, since stripping a bit changes which cell the parity bit would be, and it stands aside when the operator has forced the width. It re-decodes at the shorter width rather than shifting the values, because the shorter frame resynchronises earlier and can pick up a start edge the longer reading steps over.

Promoting 8N1 to 7E1 or 7O1. This is the one pair error scoring cannot tell apart: seven data bits plus parity occupy the same ten cells as eight data bits with the stop bit in the same place, so 7E1 traffic decodes as 8N1 with zero framing errors and wins on search order. What gives it away is that bit 7 would equal the parity of bits 0 to 6 in *every* frame — a 1-in-2 coincidence per frame.

Both refinements demand at least 6 good frames and 3 distinct values, because a repeated byte is ambiguous: 0x55 as 8N1 and as 7E1 give identical waveforms, and the common format should win. The parity vote excludes the three head frames and the last one before counting, always — a gapless stream resynchronises a few frames in, so the head is suspect whether or not the framer flagged it, and a misaligned head that happens to frame cleanly as 8N1 produces no flag at all.

Two further guards hold the vote honest. It needs at least two frames in each bit-7 state, because 48 of the 95 printable ASCII characters have even popcount — so an 8N1 payload drawn only from those shows perfect apparent 7E1 agreement over arbitrarily many distinct values while lacking the evidence to prefer it. And a vote short of unanimous must clear 95 % agreement over at least 20 frames, and forces a real re-decode rather than a rewrite in place: under a fraction, the dissenters would otherwise keep a clean bill of health while contradicting the format just assigned, so the fix itself would become a silent wrong answer.

Locking the answer

Re-deriving the rate and format costs an extra capture and the whole format search, so the app locks them once it has earned the right. Five gates, all of which must pass:

1. the rate is credible — it snapped to a standard rate, or it rounds to a boundary an operator could have typed
2. the longest *contiguous* clean run is at least 8 frames and at least 60 % of the capture
3. at least 8 good frames, because two clean frames of noise agree with each other too
4. the least-squares bit time explains the pulse widths it was fitted to, `q` ≥ 0.9
5. the baseline is stable

Gate 1 is deliberately near-vacuous. Rounding cascades down to whole baud, and half a baud is under 1 % of anything above the floor, so credibility asks "is this a rate at all" rather than "does it land on a tidy boundary". It had to: with a fixed 10 baud step, 160 of the 901 integer rates between 100 and 1000 baud were refused, so a 105-baud line could not be locked, every recording mode refused it, and the operator was back at the keypad that the rounding exists to avoid. A boundary is about presentation, and presentation was never a reason to refuse a lock. The four gates after it carry the whole judgement.

Gate 2 is a run rather than an error count because a wrong format gives *scattered* errors, which leave no long unbroken stretch however few of them there are, and a run tolerates head resync and tail clipping without having to guess where they are allowed to be. Gate 5 is the only one about the signal rather than the decode: gates 2 to 4 describe how self-consistent one capture was, and a marginal line can be self-consistent by luck.

Polarity is deliberately *not* locked. Left open it costs one comparison per capture; locked it freezes whatever the contest decided on the detect capture, which is the shortest and least evidenced one the app takes. Measured on the instrument: that pass flipped to inverted, the lock froze it, and every capture afterwards was confidently wrong at 227 frames and zero errors. It is the only one of the five fields the contest gets wrong.

When a forced rate is wrong

A forced rate the device is not using is this decoder's one dangerous failure, and it is asymmetric. The stop bit is sampled 9.5 bit times after the start edge, so accumulated error must stay inside half a cell: the cliff is 0.5/9.5 = 5.26 %. Measured at 8N1 forced to 9600, eight seeds per point, −4.5 % to +5.0 % is byte-exact. At −5 % and below the sampling point drifts past the stop bit into the next start bit, which fails the stop check and is reported. At +6 % and above it drifts *early*, staying inside the previous cell, so every frame remains internally self-consistent — nothing downstream can detect it, and the panel shows confident garbage.

So the app measures the fit even when it does not use it. The cost is one pass over the width array, the widths being built already, and it is the only evidence available that a typed number is wrong. The warning fires at 4 %, just inside the cliff: at the point where the next percent could start corrupting bytes silently, rather than after corruption that by construction leaves no other trace.

The verdict splits in two, because the fit's own bias has to be allowed for. A measured bit time near an integer multiple or sub-multiple of the forced one is usually the fit erring short — it finds the greatest common period — so judging that as disagreement would fire on most healthy captures. But it is also exactly what a device running at twice the forced rate looks like, and that decodes with no framing error and wrong bytes: force 9600, transmit 0x43 at 19200, read 0x78 cleanly. The two cannot be separated from

the ratio, so the app states the measurement instead of suppressing the warning — "bytes may be WRONG if the device runs at 19200, 2x the 9600 you set". Any other disagreement past 4 % names the measured rate directly.

Both warnings are advisory. The operator typed that rate deliberately and may be right when the fit is not: a 5N1.5 Baudot line defeats the width fit structurally, which is exactly when someone types a rate in by hand.

When the locked rate explains nothing — at least a quarter of the interior frames bad, over at least 8 frames, with the fit disagreeing — the app unlocks, decodes the same samples again, and adopts the detected rate if that succeeds. A rate that explains the capture is not a rate nobody chose, and the alternative was worse: with a doubled generator rate under a 19200 lock, repeated presses showed an all-red panel and the only route out was typing 0 into Options, which is also how a wrong rate gets there. The retry has to earn it, though — a correct typed rate with a wrong forced *format* also fails every frame, so the result is kept only if it beats the threshold that triggered the retry.

Recordings longer than memory

A recording larger than one buffer is decoded window by window, and the split follows a distinction in the statistics: levels are an amplitude statistic, edges are a time statistic.

`sig_levels` never looks at a sample's neighbours — it takes the extremes, bins amplitudes, walks the two densest humps and averages inside them. All of that is order-free, so the levels are measured once, on a decimated view spanning the whole recording. That costs one window of memory and sees exactly the population a monolithic decode would have seen. Averaging over the whole recording also absorbs slow drift better than any one window can, since N full drift cycles are symmetric about the true levels and a quarter of one is not. Chunked mode therefore measures the line *better* than frame mode, whose capture is one window.

Decimation has one blind spot. A short message in a long recording — exactly the traffic these modes exist for — can reduce to a handful of samples at the space level once the step is 140, or to none, so a decimated view aliases past the burst and reports a measurable line as idle. When it finds nothing the app falls back to eight contiguous probe windows spaced evenly across the recording.

Re-measuring the levels per window would be the opposite error. A window that is entirely idle — normal on a bursty line, which is the traffic these modes exist for — has only one dense amplitude, so it would refuse, or split that hump and place the threshold inside a logic level, and the bytes on either side of the seam would come from two different thresholds.

Edges cannot be treated that way, because a threshold crossing is defined between adjacent samples, so they are rebuilt per window. Windows overlap by three frames plus the crossing walk-back, and the framer resumes at the position where the previous frame ended — which is simultaneously the continuation rule and the de-duplication rule for the overlap, so the seam falls in the middle of the overlap rather than at its rim. The stride is constant, which costs about 1.6 % in redundant decoding and buys a window count known before the run starts.

Declared limits

limit	value	basis
samples per bit	4	below this the mid-bit sampling point cannot be held across a ten-bit frame

limit	value	basis
bit rate ceiling	250 kBd	4 samples per bit at the top sample rate, and where the harness finds a clean waveform stops being reliable
bit rate floor	100 baud	a scope decision: the standard ladder starts at 110, and the telex rates below it cannot be locked
swing floor	0.1 V	below it there is nothing to place a threshold between

The ceiling is declared rather than emergent, and the gap between the two measurements is the reason. A clean *synthetic* waveform decodes well past it — the harness recovers 13 of 13 bytes at 2.5 samples per bit — while a real line at the bench degrades around 250 kBd.

The samples-per-bit floor and both baud limits carry 2 % slack. A genuine 250 kBd line measures 3.99998 samples per bit, and refusing it for being two hundred-thousandths under its own floor would be a bug rather than a limit; the floor needs the same slack for the mirror case, a 50 baud line whose fit measured 100.05 samples and computed to 49.975 baud. The swing floor carries none.

What the app will not guess

The app honours three settings when given - it won't search for them.

A **second stop bit** is indistinguishable from a bit time of idle, which the framer already tolerates, so searching for it could only tie. A 2-stop line decodes correctly and is reported as 1 stop.

Nine data bits is not searched because biasing against a wide frame does not work: the bias scales the proportional term of the margin, which is zero exactly when the honest reading is damaged — and that is exactly when a laundering width can win. Measured on an 8N1 line with stop-bit damage, 9N1 returned `nbad = 0` with 9 of 12 bytes wrong, every one 256 too large.

Polarity is always decided, never refused. Where the capture holds no idle the decision comes from the signalling standard rather than from the capture, and the app marks that prior weak and raises the bar a rival must clear. Options → Invert overrides it.

What unites the three is the design rule the decoder is built on: a refusal names the limit it applied and an assumption is stated on the panel - the failure this code spends most of its effort avoiding is not a refusal — it's a wrong answer that looks right.